Network Data Simulation

Network Data Simulation

```r
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(igraph)
```

```
##
## Attaching package: 'igraph'

## The following objects are masked from 'package:dplyr':
##
##     as_data_frame, groups, union

## The following objects are masked from 'package:stats':
##
##     decompose, spectrum

## The following object is masked from 'package:base':
##
##     union
```

Build network data function

```r
build_network <- function(x1, x2) {

  edges <- data.frame(
    EGO_ID   = as.character(x1),
```

```r
    ALTER_ID = as.character(x2)
  )

  nodes <- data.frame(
    name = unique(c(edges$EGO_ID, edges$ALTER_ID))
  )

  net0 <- igraph::graph_from_data_frame(
    d = edges,
    vertices = nodes,
    directed = FALSE
  )

  net0 <- igraph::delete_vertices(net0, which(igraph::degree(net0) == 0))
  net0 <- igraph::simplify(net0, remove.multiple = TRUE, remove.loops = TRUE)

  n <- igraph::vcount(net0)

  data <- data.frame(
    subject_id = as.integer(igraph::V(net0)$name),
    id = seq_len(n)
  )

  data$na <- igraph::degree(net0)

  return(list(
    net0 = net0,
    n = n,
    data = data
  ))
}
```

Simulate Network Outcome Function

```r
simulate_network_outcome <- function(net_obj,
                                     p_L = 0.4,
                                     outcome_intercept = 0.5,
                                     beta_a = -0.5,
                                     beta_abar = -0.5,
                                     beta_a_abar = 0.25,
                                     beta_l = 0.25,
                                     propensity_intercept = -0.5,
                                     propensity_L = 1,
                                     define_observed_outcome = FALSE) {

  net0 <- net_obj$net0
  dat.network <- net_obj$data

  # rename
  dat.network$interference_set_size <- dat.network$na
```

```r
# generate covariate
dat.network$L_i <- rbinom(nrow(dat.network), 1, p_L)

data <- dat.network

n <- nrow(data)

Ni <- data$interference_set_size
Li <- data$L_i

PO <- vector("list", length = n)

for (i in seq_len(n)) {
  PO[[i]] <- matrix(NA_integer_, nrow = 2, ncol = Ni[i] + 1)
}

for (i in seq_len(n)) {
  for (j in 0:1) {
    for (k in 0:Ni[i]) {

      abar <- k / Ni[i]

      eta <- outcome_intercept +
        beta_a * j +
        beta_abar * abar +
        beta_a_abar * j * abar +
        beta_l * Li[i]

      p <- plogis(eta)

      PO[[i]][j + 1, k + 1] <- rbinom(1, 1, p)

    }
  }
}

if (!define_observed_outcome) {
  return(PO)
}

# treatment assignment
n_net <- nrow(dat.network)

dat.network$Ai <- rbinom(
  n_net,
  size = 1,
  prob = plogis(propensity_intercept + propensity_L * dat.network$L_i)
)

# neighbor treatment exposure
dat.network$Abar_Ni <- vapply(
  seq_len(nrow(dat.network)),
  function(i) {
```

```r
      nbrs <- igraph::neighbors(net0, i)

      mean(dat.network$Ai[as.integer(nbrs)])

    },
    numeric(1)
  )

  # observed outcome
  dat.network$Y_obs <- vapply(
    seq_len(nrow(dat.network)),
    function(i) {

      PO_i <- PO[[i]]

      PO_i[
        dat.network$Ai[i] + 1,
        dat.network$Abar_Ni[i] + 1
      ]

    },
    numeric(1)
  )

  dat.network$na <- NULL

  return(dat.network)
}
```

Construct network data

```r
# Example edge list
x1 <- c(1, 1, 2, 3, 4, 5)
x2 <- c(2, 3, 3, 4, 5, 6)

net_obj <- build_network(x1, x2)

net_obj$data
```

```
##   subject_id id na
## 1          1  1  2
## 2          2  2  2
## 3          3  3  3
## 4          4  4  2
## 5          5  5  2
## 6          6  6  1
```

Description

The `build_network()` function constructs an undirected network from an edge list.

The returned object contains three components:

- `net0`: the simplified undirected `igraph` object
- `n`: the number of nodes in the network
- `data`: a data frame containing node-level information

The node-level dataset includes the following variables:

- `subject_id`: the original node label from the input edge list
- `id`: the internal node index in the network
- `na`: the number of neighbors for each node

For each individual i, the interference set is defined as the set of all directly connected neighbors in the network.

Generate potential outcomes

```
PO <- simulate_network_outcome(
  net_obj,
  define_observed_outcome = FALSE
)

length(PO)
```

```
## [1] 6
```

```
PO
```

```
## [[1]]
##      [,1] [,2] [,3]
## [1,]    1    1    1
## [2,]    0    1    1
##
## [[2]]
##      [,1] [,2] [,3]
## [1,]    0    0    1
## [2,]    0    0    1
##
## [[3]]
##      [,1] [,2] [,3] [,4]
## [1,]    0    0    0    0
## [2,]    0    1    0    0
##
## [[4]]
##      [,1] [,2] [,3]
## [1,]    1    1    0
```

```
## [2,]    0    1    0
##
## [[5]]
##      [,1] [,2] [,3]
## [1,]    1    1    1
## [2,]    0    1    0
##
## [[6]]
##      [,1] [,2]
## [1,]    1    1
## [2,]    1    0
```

Description

For each individual i, the function generates a matrix of potential outcomes.

- Rows correspond to individual treatment values $a \in \{0, 1\}$
- Columns correspond to the number of treated neighbors $k \in \{0, \ldots, N_i\}$

Thus, the entry

$$PO[[i]][a+1, k+1]$$

represents the potential outcome for individual i under treatment a and exactly k treated neighbors in the interference set.

The outcome probability is generated from the logistic model

$$\text{logit}\{P(Y_i = 1)\} = \beta_0 + \beta_a a + \beta_{\bar{a}} \bar{a} + \beta_{a\bar{a}}(a \times \bar{a}) + \beta_l L_i,$$

where

$$\bar{a} = \frac{k}{N_i}.$$

Here, L_i is a binary baseline covariate, $\beta_0 = 0.5$, $\beta_a = -0.5$, $\beta_{\bar{a}} = -0.5$, $\beta_{a\bar{a}} = 0.25$, and $\beta_l = 0.25$.

Define observed treatment and observed outcome

```r
dat.obs <- simulate_network_outcome(
  net_obj,
  define_observed_outcome = TRUE
)

head(dat.obs)
```

```
##   subject_id id interference_set_size L_i Ai   Abar_Ni Y_obs
## 1          1  1                     2   0  0 0.5000000     0
## 2          2  2                     2   0  1 0.0000000     0
```

```
## 3          3  3                     3   0  0 0.6666667     1
## 4          4  4                     2   0  1 0.5000000     1
## 5          5  5                     2   0  1 0.5000000     0
## 6          6  6                     1   0  0 1.0000000     1
```

Description

When `define_observed_outcome = TRUE`, the function additionally generates observed treatment, neighborhood exposure, and observed outcomes.

Individual treatment is assigned according to the propensity score model

$$P(A_i = 1 \mid L_i) = \text{expit}(-0.5 + L_i).$$

For each individual i, the neighborhood treatment exposure is computed as the average treatment among all neighbors:

$$\bar{A}_{N_i} = \frac{1}{N_i} \sum_{j \in N(i)} A_j,$$

where $N(i)$ denotes the set of neighbors of individual i.

The observed outcome is then defined by consistency as

$$Y_i^{\text{obs}} = Y_i(A_i, \bar{A}_{N_i}).$$

In the implementation, this is obtained by selecting the entry indexed by the observed treatment and observed neighborhood exposure from the potential outcome matrix.

```
PO_i[
  dat.obs$Ai[i] + 1,
  dat.obs$Abar_Ni[i] + 1
]
```

Output

Potential outcome output

When `define_observed_outcome = FALSE`, the function returns a list of potential outcome matrices:

```
class(PO)
```

```
## [1] "list"
```

```
length(PO)
```

```
## [1] 6
```

Observed data output

When `define_observed_outcome = TRUE`, the function returns a data frame containing the observed data:

```
colnames(dat.obs)
```

```
## [1] "subject_id"            "id"                    "interference_set_size"
## [4] "L_i"                   "Ai"                    "Abar_Ni"
## [7] "Y_obs"
```

The returned data frame includes:

- `subject_id`
- `id`
- `interference_set_size`
- `L_i`
- `Ai`
- `Abar_Ni`
- `Y_obs`

Example

```
x1 <- c(1, 1, 2, 3, 4, 5)
x2 <- c(2, 3, 3, 4, 5, 6)

net_obj <- build_network(x1, x2)

# potential outcomes only
PO <- simulate_network_outcome(
  net_obj,
  define_observed_outcome = FALSE
)

# observed data
obs.dat <- simulate_network_outcome(
  net_obj,
  define_observed_outcome = TRUE
)

head(obs.dat)
```

```
##   subject_id id interference_set_size L_i Ai   Abar_Ni Y_obs
## 1          1  1                     2   1  1 0.5000000     0
## 2          2  2                     2   1  1 0.5000000     1
## 3          3  3                     3   1  0 0.6666667     1
```

```
## 4          4  4                    2   0  0 0.5000000     1
## 5          5  5                    2   1  1 0.5000000     1
## 6          6  6                    1   1  1 1.0000000     1
```